

UNIT 3: PROCESS SYNCHRONIZATION

Process Synchronization is the coordination of execution of multiple processes in a multi-process system to ensure that they access shared resources in a controlled and predictable manner. It aims to resolve the problem of race conditions and other synchronization issues in a concurrent system.

The main objective of process synchronization is to ensure that multiple processes access shared resources without interfering with each other, and to prevent the possibility of inconsistent data due to concurrent access. To achieve this, various synchronization techniques such as semaphores, monitors, and critical sections are used.

In a multi-process system, synchronization is necessary to ensure data consistency and integrity, and to avoid the risk of deadlocks and other synchronization problems. Process synchronization is an important aspect of modern operating systems, and it plays a crucial role in ensuring the correct and efficient functioning of multi-process systems.

On the basis of synchronization, processes are categorized as one of the following two types:

- **Independent Process:** The execution of one process does not affect the execution of other processes.
- **Cooperative Process:** A process that can affect or be affected by other processes executing in the system.

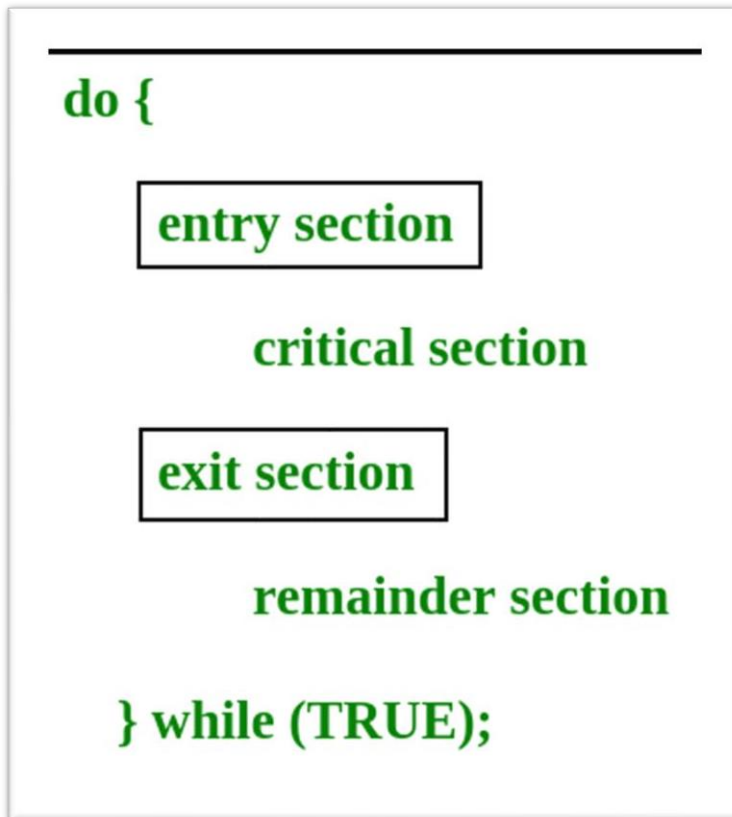
Process synchronization problem arises in the case of Cooperative process also because resources are shared in Cooperative processes.

Race Condition

When more than one process is executing the same code or accessing the same memory or any shared variable in that condition there is a possibility that the output or the value of the shared variable is wrong so for that all the processes doing the race to say that my output is correct this condition known as a race condition. Several processes access and process the manipulations over the same data concurrently, then the outcome depends on the particular order in which the access takes place. A race condition is a situation that may occur inside a critical section. This happens when the result of multiple thread execution in the critical section differs according to the order in which the threads execute. Race conditions in critical sections can be avoided if the critical section is treated as an atomic instruction. Also, proper thread synchronization using locks or atomic variables can prevent race conditions.

Critical Section Problem

A critical section is a code segment that can be accessed by only one process at a time. The critical section contains shared variables that need to be synchronized to maintain the consistency of data variables. So the critical section problem means designing a way for cooperative processes to access shared resources without creating data inconsistencies.



In the entry section, the process requests for entry in the **Critical Section**.

Any solution to the critical section problem must satisfy three requirements:

- **Mutual Exclusion:** If a process is executing in its critical section, then no other process is allowed to execute in the critical section.
- **Progress:** If no process is executing in the critical section and other processes are waiting outside the critical section, then only those processes that are not executing in their remainder section can participate in deciding which will enter in the critical section next, and the selection can not be postponed indefinitely.
- **Bounded Waiting:** A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Peterson's Solution

Peterson's Solution is a classical software-based solution to the critical section problem. In Peterson's solution, we have two shared variables:

- boolean flag[i]: Initialized to FALSE, initially no one is interested in entering the critical section
- int turn: The process whose turn is to enter the critical section.

```

do {

    flag[i] = TRUE ;
    turn = j ;
    while (flag[j] && turn == j) ;

    critical section

    flag[i] = FALSE ;

    remainder section

} while (TRUE) ;

```

Peterson's Solution is a classic software-based solution to the critical section problem. It is unfortunately not guaranteed to work on modern hardware, but it illustrates a number of important concepts.

Peterson's solution is based on two processes, P₀ and P₁, which alternate between their critical sections and remainder sections. For convenience of discussion, "this" process is P_i, and the "other" process is P_j. (i.e. $j = 1 - i$)

Peterson's solution requires two shared data items:

- **int turn** - Indicates whose turn it is to enter into the critical section. If $\text{turn} = i$, then process i is allowed into their critical section.
- **boolean flag[2]** - Indicates when a process *wants to* enter into their critical section. When process i wants to enter their critical section, it sets $\text{flag}[i]$ to true.

In the following diagram, the entry and exit sections are enclosed in boxes.

- In the entry section, process i first raises a flag indicating a desire to enter the critical section.
- Then turn is set to j to allow the *other* process to enter their critical section *if process j so desires*.
- The while loop is a busy loop (notice the semicolon at the end), which makes process i wait as long as process j has the turn and wants to enter the critical section.
- Process i lowers the $\text{flag}[i]$ in the exit section, allowing process j to continue if it has been waiting.

do {

```
flag[i] = TRUE;
turn = j;
while (flag[j] && turn == j);
```

critical section

```
flag[i] = FALSE;
```

remainder section

} while (TRUE);

To prove that the solution is correct, we must examine the three conditions listed above:

1. **Mutual exclusion** - If one process is executing their critical section when the other wishes to do so, the second process will become blocked by the flag of the first process. If both processes attempt to enter at the same time, the last process to execute " $\text{turn} = j$ " will be blocked.
2. **Progress** - Each process can only be blocked at the while if the other process wants to use the critical section ($\text{flag}[j] == \text{true}$), AND it is the other process's turn to use the critical section ($\text{turn} == j$). If both of those conditions are true, then the other process (j) will be allowed to enter the critical section, and upon exiting the critical section,

will set `flag[j]` to false, releasing process `i`. The shared variable `turn` assures that only one process at a time can be blocked, and the `flag` variable allows one process to release the other when exiting their critical section.

3. **Bounded Waiting** - As each process enters their entry section, they set the `turn` variable to be the other processes' turn. Since no process ever sets it back to their own turn, this ensures that each process will have to let the other process go first at most one time before it becomes their turn again.

Note that the instruction "`turn = j`" is *atomic*, that is it is a single machine instruction which cannot be interrupted.

Peterson's Solution preserves all three conditions:

- Mutual Exclusion is assured as only one process can access the critical section at any time.
- Progress is also assured, as a process outside the critical section does not block other processes from entering the critical section.
- Bounded Waiting is preserved as every process gets a fair chance.

Disadvantages of Peterson's solution:

- It involves busy waiting. (In the Peterson's solution, the code statement- "`while(flag[j] && turn == j);`" is responsible for this. Busy waiting is not favored because it wastes CPU cycles that could be used to perform other tasks.)
- It is limited to 2 processes.
- Peterson's solution cannot be used in modern CPU architectures.

Synchronization Hardware

To generalize the solution(s) expressed above, each process when entering their critical section must set some sort of *lock*, to prevent other processes from entering their critical sections simultaneously, and must release the lock when exiting their critical section, to allow other processes to proceed. Obviously it must be possible to attain the lock only when no other process has already set a lock. Specific implementations of this general procedure can get quite complicated, and may include hardware solutions as outlined in this section.

Test and Set

One simple solution to the critical section problem is to simply prevent a process from being

interrupted while in their critical section, which is the approach taken by non preemptive kernels. Unfortunately, this does not work well in multiprocessor environments, due to the difficulties in disabling and the re-enabling interrupts on all processors. There is also a question as to how this approach affects timing if the clock interrupt is disabled.

Another approach is for hardware to provide certain *atomic* operations. These operations are guaranteed to operate as a single instruction, without interruption. One such operation is the "Test and Set", which simultaneously sets a Boolean lock variable and returns its previous value

```
boolean TestAndSet(boolean *target) {
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

The definition of the TestAndSet () instruction.

```
do {
    while (TestAndSetLock(&lock))
        ; // do nothing

    // critical section

    lock = FALSE;

    // remainder section
}while (TRUE);
```

Mutual-exclusion implementation with TestAndSet ().

Compare and Swap

Another variation on the test-and-set is an atomic swap of two Booleans variables which acts as a lock and key. This swaps contents of two memory words and maintain mutual exclusion in accessing them.

```
void Swap (boolean *a, boolean *b)
{
    boolean temp = *a;
    *a = *b;
    *b = temp;
}
```

Definition of Compare and Swap

```
do {
    key = TRUE;
    while ( key == TRUE)
        swap (&lock, &key ); (key=F only then enters the cs)
        // critical section
    lock = FALSE;
        // remainder section
} while (TRUE);
```

Semaphores

Semaphores can also be used to synchronize certain operations between processes.

Wait:

```
wait(S) {
    while S <= 0
        ; // no-op
    S--;
}
```

Signal:

```
signal(S) {
    S++;
}
```

Semaphores are more robust to use *and it is just an* integer variable for which only two atomic operations are defined, the wait and signal operations, as shown in the following figure. Note that not only must the variable-changing steps (S-- and S++) be indivisible, it is also necessary that for the wait operation when the test proves false that there be no interruptions before S gets decremented. It is okay, however, for the busy loop to be interrupted when the test is true, which

prevents the system from hanging forever.

In practice, semaphores can take on one of two forms:

- 1) **Binary semaphores** can take on one of two values, 0 or 1.
- 2) **Counting semaphores** can take on any integer value, and are usually used to count the number remaining of some limited resource. The counter is initialized to the number of such resources available in the system, and whenever the counting semaphore is greater than zero, then a process can enter a critical section and use one of the resources. When the counter gets to zero (or negative in some implementations), then the process blocks until another process frees up a resource and increments the counting semaphore with a signal call. (The binary semaphore can be seen as just a special case where the number of resources initially available is just one.)

Classic problem of synchronization

Bounded Buffer Problem

This is a generalization of the producer-consumer problem wherein access is controlled to a shared group of buffers of a limited size.

In this solution, the two counting semaphores "full" and "empty" keep track of the current number of full and empty buffers respectively (and initialized to 0 and N respectively.) The binary semaphore mutex controls access to the critical section. The producer and consumer processes are nearly identical - One can think of the producer as producing full buffers, and the consumer producing empty buffers.


```

do {
    . . .
    // produce an item in nextp
    . . .
    wait(empty);
    wait(mutex);
    . . .
    // add nextp to buffer
    . . .
    signal(mutex);
    signal(full);
}while (TRUE);

```

The structure of the producer process.

```

do {
    wait(full);
    wait(mutex);
    . . .
    // remove an item from buffer to nextc
    . . .
    signal(mutex);
    signal(empty);
    . . .
    // consume the item in nextc
    . . .
}while (TRUE);

```

The structure of the consumer process.

Readers Writers Problem

- In the readers-writers problem there are some processes (termed readers) who only read the shared data, and never change it, and there are other processes (termed writers) who may change the data in addition to or instead of reading it. There is no limit to how many readers can access the data simultaneously, but when a writer accesses the data, it needs exclusive access.
- There are several variations to the readers-writers problem, most centered around relative priorities of readers versus writers.

- The *first* readers-writers problem gives priority to readers. In this problem, if a reader wants access to the data, and there is not already a writer accessing it, then access is granted to the reader. A solution to this problem can lead to starvation of the writers, as there could always be more readers coming along to access the data.

```
do {
    wait(rw_mutex);

    . . .
    /* writing is performed */
    . . .
    signal(rw_mutex);
} while (true);
```

The structure of a writer process.

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    . . .
    /* reading is performed */
    . . .
    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

The structure of a reader process.

- The *second* readers-writers problem gives priority to the writers. In this problem, when a writer wants access to the data it jumps to the head of the queue - All waiting readers are blocked, and the writer gets access to the data as soon as it becomes available. In this solution the readers may be starved by a steady stream of writers.

The above code is an example of the first readers-writers problem, and involves an important counter and two binary semaphores:

- readcount is used by the reader processes, to count the number of readers currently accessing the data.
- mutex is a semaphore used only by the readers for controlled access to readcount.
- rw_mutex is a semaphore used to block and release the writers. The first reader to access the data will set this lock and the last reader to exit will release it; The remaining readers do not touch rw_mutex. (Eighth edition called this variable wrt.)
- Note that the first reader to come along will block on rw_mutex if there is currently a writer accessing the data, and that all following readers will only block on mutex for their turn to increment readcount.

Dining Philosophers Problem

The dining philosopher's problem is a classic synchronization problem involving the allocation of limited resources amongst a group of processes in a deadlock-free and starvation-free manner: Consider five philosophers sitting around a table, in which there are five chopsticks evenly distributed and an endless bowl of rice in the center, as shown in the diagram below. (There is exactly one chopstick between each pair of dining philosophers). These philosophers spend their lives alternating between two activities: eating and thinking.



When it is time for a philosopher to eat, it must first acquire two chopsticks - one from their left and one from their right. When a philosopher thinks, it puts down both chopsticks in their original locations.

- One possible solution, as shown in the following code section, is to use a set of five semaphores (chopsticks[5]), and to have each hungry philosopher first wait on their left chopstick (chopsticks[i]), and then wait on their right chopstick (chopsticks[(i + 1) % 5])
- But suppose that all five philosophers get hungry at the same time, and each starts by picking up their left chopstick. They then look for their right chopstick, but because it is unavailable, they wait for it, forever, and eventually all the philosophers starve due to the resulting deadlock

```
do {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);
    . . .
    // eat
    . . .
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    . . .
    // think
    . . .
}while (TRUE);
```

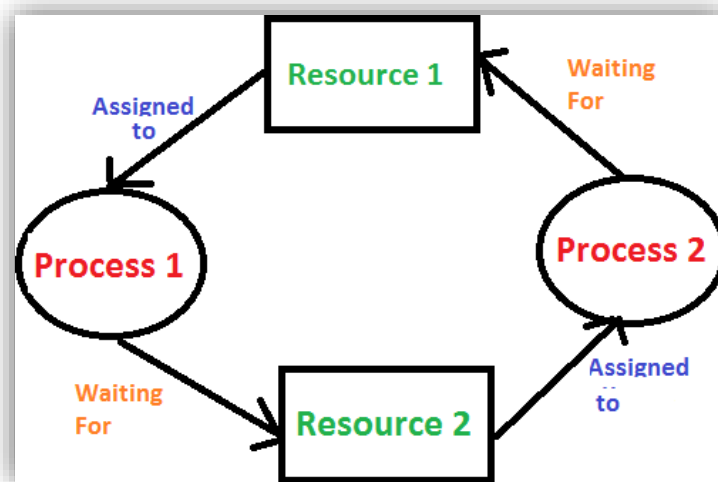
Some potential solutions to the problem include:

- Only allow four philosophers to dine at the same time.
- Allow philosophers to pick up chopsticks only when both are available, in a critical section.
- Use an asymmetric solution, in which odd philosophers pick up their left chopstick first and even philosophers pick up their right chopstick first.

Deadlock System model

The computer system consists of finite number of resources which has to be shared among the competing process. The resources can be CPU cycles, files, memory space, I/O devices etc. Some of the resources can be single and some might be multiple. Each of resource of the same type is called as an instance. The resources can be represented as $R_1, R_2 \dots R_m$ and Each resource can W_i instances. Each process request for a resource, use the resource and release the resource.

Deadlock happens in operating system when two or more processes need some resource to complete their execution that is held by the other process. Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

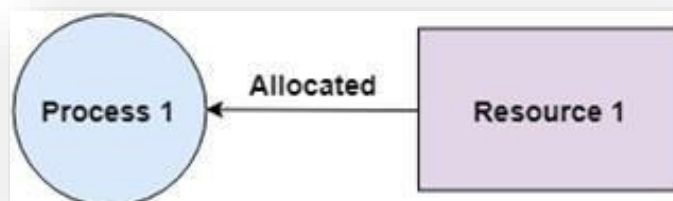


Deadlock Characterization

The following are the necessary conditions for the deadlock. **When all of the 4 conditions occur simultaneously then deadlock occurs.**

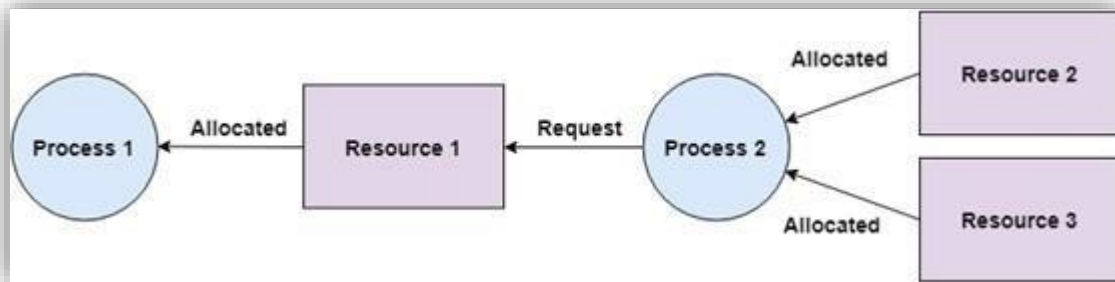
1) Mutual Exclusion

There should be a resource that can only be held by one process at a time. In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.



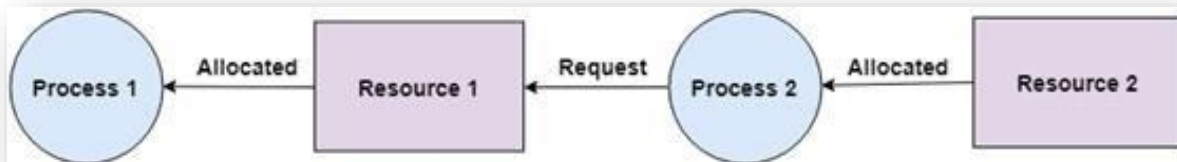
2) Hold and Wait

A process can hold multiple resources and still request more resources from other processes which are holding them. In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.



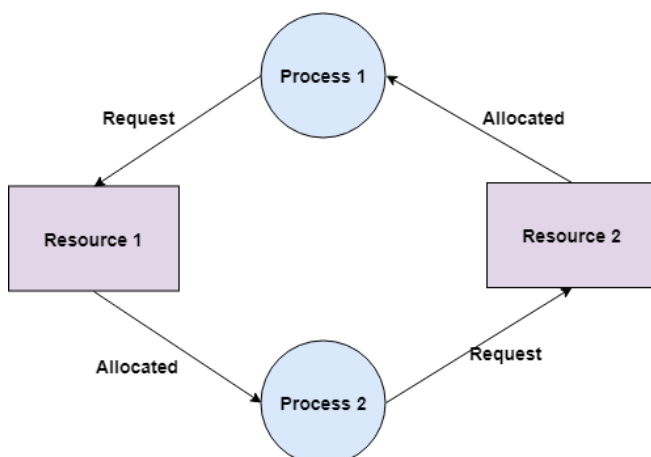
3) No Preemption

A resource cannot be preempted from a process by force. A process can only release a resource voluntarily. In the diagram below, Process 2 cannot preempt Resource 1 from Process 1. It will only be released when Process 1 relinquishes it voluntarily after its execution is complete.



4) Circular Wait

A process is waiting for the resource held by the second process, which is waiting for the resource held by the third process and so on, till the last process is waiting for a resource held by the first process. This forms a circular chain. For example: Process 1 is allocated Resource 2 and it is requesting Resource 1. Similarly, Process 2 is allocated Resource 1 and it is requesting Resource 2. This forms a circular wait loop.



Methods of Handling Deadlock

Generally speaking there are three ways of handling deadlocks:

- 1) Deadlock prevention or avoidance - Do not allow the system to get into a deadlocked state.
- 2) Deadlock detection and recovery - Abort a process or preempt some resources when deadlocks are detected.
- 3) Ignore the problem all together - If deadlocks only occur once a year or so, it may be better to simply let them happen and reboot as necessary than to incur the constant overhead and system performance penalties associated with deadlock prevention or detection. This is the approach that both Windows and UNIX take.

In order to avoid deadlocks, the system must have additional information about all processes. In particular, the system must know what resources a process will or may request in the future.

(Ranging from a simple worst-case maximum to a complete resource request and release plan for each process, depending on the particular algorithm.)

Deadlock detection is fairly straightforward, but deadlock recovery requires either aborting processes or preempting resources, neither of which is an attractive alternative

If deadlocks are neither prevented nor detected, then when a deadlock occurs the system will gradually slow down, as more and more processes become stuck waiting for resources currently held by the deadlock and by other waiting processes. Unfortunately, this slow down can be indistinguishable from a general system slowdown when a real-time process has heavy computing needs.

Deadlock prevention

Deadlocks can be prevented by preventing at least one of the four required conditions from being met.

Mutual Exclusion

Shared resources such as read-only files do not lead to deadlocks.

Unfortunately, some resources, such as printers and tape drives, require exclusive access by a single process.

Hold and Wait

To prevent this condition processes must be prevented from holding one or more resources while simultaneously waiting for one or more others. There are several possibilities for this:

- Require that all processes request all resources at one time. This can be wasteful of system resources if a process needs one resource early in its execution and doesn't need some other resource until much later.

- Require that processes holding resources must release them before requesting new resources, and then re-acquire the released resources along with the new ones in a single new request. This can be a problem if a process has partially completed an operation using a resource and then fails to get it re-allocated after releasing it.
- Either of the methods described above can lead to starvation if a process requires one or more popular resources.

No Preemption

Preemption of process resource allocations can prevent this condition of deadlocks, when it is possible.

- One approach is that if a process is forced to wait when requesting a new resource, then all other resources previously held by this process are implicitly released, (preempted), forcing this process to re-acquire the old resources along with the new resources in a single request, similar to the previous discussion.
- Another approach is that when a resource is requested and not available, then the system looks to see what other processes currently have those resources *and* are themselves blocked waiting for some other resource. If such a process is found, then some of their resources may get preempted and added to the list of resources for which the process is waiting.
- Either of these approaches may be applicable for resources whose states are easily saved and restored, such as registers and memory, but are generally not applicable to other devices such as printers etc.

Circular Wait

One way to avoid circular wait is to number all resources, and the processes has to request resources only in strictly increasing (or decreasing) order.

- In other words, in order to request resource R_j , a process must first release all R_i such that $i \geq j$.
- One big challenge in this scheme is determining the relative ordering of the different resources.
- The general idea behind deadlock avoidance is to prevent deadlocks from ever happening, by preventing at least one of the aforementioned conditions. This requires more information about each process, AND tends to lead to low device utilization. In some

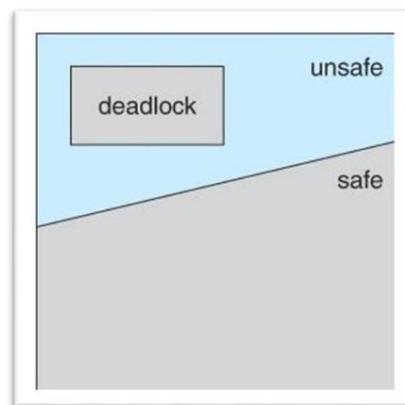
algorithms the scheduler only needs to know the *maximum* number of each resource that a process might potentially use. In more complex algorithms the scheduler can also take advantage of the *schedule* of exactly what resources may be needed in what order. When a scheduler sees that starting a process or granting resource requests may lead to future deadlocks, then that process is just not started or the request is not granted.

- A resource allocation **state** is defined by the number of available and allocated resources, and the maximum requirements of all processes in the system.

Safe State

- A state is **safe** if the system can allocate all resources requested by all processes without entering a deadlock state.
- More formally, a state is safe if there exists a **safe sequence** of processes $\{ P_0, P_1, P_2, \dots, P_N \}$ such that all of the resource requests for P_i can be granted using the resources currently allocated to P_i and all processes P_j where $j < i$. (i.e. if all the processes prior to P_i finish and free up their resources, then P_i will be able to finish also, using the resources that they have freed up.

If a safe sequence does not exist, then the system is in an unsafe state, which **MAY** lead to deadlock.

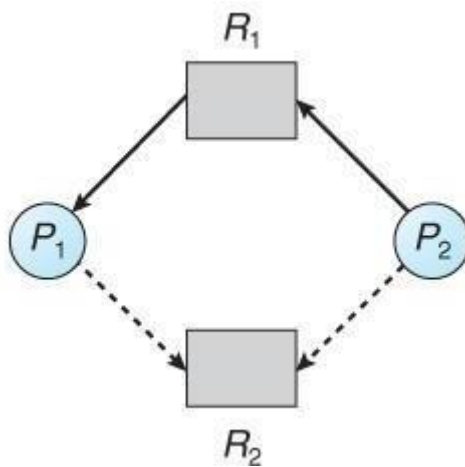


Deadlock Avoidance for Single Resource

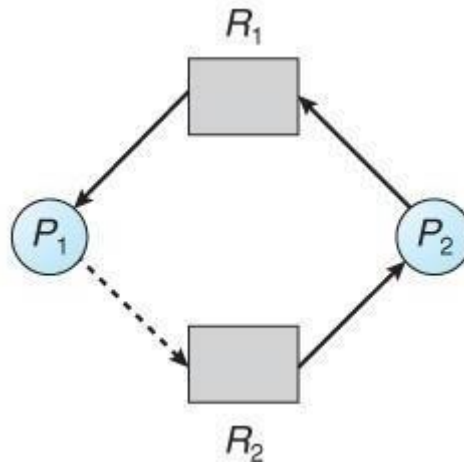
The deadlock can be avoided for a single resource instance using Resource allocation graph.

- In this case, unsafe states can be recognized and avoided by augmenting the resource-allocation graph with **claim edges**, noted by dashed lines, which point from a process to a resource that it may request in the future.
- In order for this technique to work, all claim edges must be added to the graph for any particular process before that process is allowed to request any resources. (Alternatively, processes may only make requests for resources for which they have already established claim edges, and claim edges cannot be added to any process that is currently holding resources.)
- When a process makes a request, the claim edge $P_i \rightarrow R_j$ is converted to a request edge. Similarly when a resource is released, the assignment reverts back to a claim edge.

This approach works by denying requests that would produce cycles in the resource-allocation graph, taking claim edges into effect. Consider for example what happens when process P_2 requests resource R_2 :



The resulting resource-allocation graph would have a cycle in it, and so the request cannot be granted.



Deadlock Avoidance for Multiple Resources

The banker's algorithm is a resource allocation and deadlock avoidance algorithm that tests for safety by simulating the allocation for predetermined maximum possible amounts of all resources, then makes an "safe-state" check to test for possible activities, before deciding whether allocation should be allowed to continue.

Why Banker's algorithm is named so?

Banker's algorithm is named so because it is used in banking system to check whether loan can be sanctioned to a person or not. Suppose there are n number of account holders in a bank and the total sum of their money is S . If a person applies for a loan, then the bank first subtracts the loan amount from the total money that bank has and if the remaining amount is greater than S then only the loan is sanctioned. It is done because if all the account holders come to withdraw their money then the bank can easily do it.

In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers. The bank would try to be in safe state always.

Following Data structures are used to implement the Banker's Algorithm:

Let ' n ' be the number of processes in the system and ' m ' be the number of resources types

Available :

- It is a 1-d array of size ' m ' indicating the number of available resources of each type.
- $\text{Available}[j] = k$ means there are ' k ' instances of resource type R_j

Max:

- It is a 2-d array of size 'n*m' that defines the maximum demand of each process in a system.
- $\text{Max}[i, j] = k$ means process P_i may request at most 'k' instances of resource type R_j .

Allocation:

- It is a 2-d array of size 'n*m' that defines the number of resources of each type currently allocated to each process.
- $\text{Allocation}[i, j] = k$ means process P_i is currently allocated 'k' instances of resource type R_j

Need:

- It is a 2-d array of size 'n*m' that indicates the remaining resource need of each process.
- $\text{Need}[i, j] = k$ means process P_i currently need 'k' instances of resource type R_j for its execution.
- $\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$

Allocation_i specifies the resources currently allocated to process P_i and Need_i specifies the additional resources that process P_i may still request to complete its task.

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

- 1) Let Work and Finish be vectors of length 'm' and 'n' respectively.

Initialize: $\text{Work} = \text{Available}$

$\text{Finish}[i] = \text{false}$; for $i=1, 2, 3, 4, \dots, n$

- 2) Find an i such that both

a) $\text{Finish}[i] = \text{false}$

b) $\text{Need}_i \leq \text{Work}$

if no such i exists goto step (4)

3) $Work = Work + Allocation[i]$

$Finish[i] = true$

goto step (2)

4) if $Finish[i] = true$ for all I , then the system is in a safe state

Example:

Considering a system with five processes P_0 through P_4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t_0 following snapshot of the system has been taken:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Need Matrix

$Need[i, j] = Max[i, j] - Allocation[i, j]$

So, the content of Need Matrix is:

Process	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Safe Sequence

$m=3, n=5$ Step 1 of Safety Algo

Work = Available

Work =

3	3	2
---	---	---

0 1 2 3 4

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

For $i=0$ Step 2

Need₀ = 7, 4, 3

Finish [0] is false and Need₀ > Work

So P₀ must wait

But Need ≤ Work

For $i=1$ Step 2

Need₁ = 1, 2, 2

Finish [1] is false and Need₁ < Work

So P₁ must be kept in safe sequence

3, 3, 2 2, 0, 0 Step 3

Work = Work + Allocation₁

Work =

5	3	2
---	---	---

0 1 2 3 4

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

For $i=2$ Step 2

Need₂ = 6, 0, 0

Finish [2] is false and Need₂ > Work

So P₂ must wait

For $i=3$ Step 2

Need₃ = 0, 1, 1

Finish [3] = false and Need₃ < Work

So P₃ must be kept in safe sequence

5, 3, 2 2, 1, 1 Step 3

Work = Work + Allocation₃

Work =

7	4	3
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	false
-------	------	-------	------	-------

For $i=4$ Step 2

Need₄ = 4, 3, 1

Finish [4] = false and Need₄ < Work

So P₄ must be kept in safe sequence

7, 4, 3 0, 0, 2 Step 3

Work = Work + Allocation₄

Work =

7	4	5
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	true
-------	------	-------	------	------

For $i=0$ Step 2

Need₀ = 7, 4, 3

Finish [0] is false and Need < Work

So P₀ must be kept in safe sequence

7, 4, 5 0, 1, 0 Step 3

Work = Work + Allocation₀

Work =

7	5	5
---	---	---

0 1 2 3 4

Finish =

true	true	false	true	true
------	------	-------	------	------

For $i=2$ Step 2

Need₂ = 6, 0, 0

Finish [2] is false and Need₂ < Work

So P₂ must be kept in safe sequence

7, 5, 5 3, 0, 2 Step 3

Work = Work + Allocation₂

Work =

10	5	7
----	---	---

0 1 2 3 4

Finish =

true	true	true	true	true
------	------	------	------	------

Finish [i] = true for $0 \leq i \leq n$ Step 4

Hence the system is in Safe state

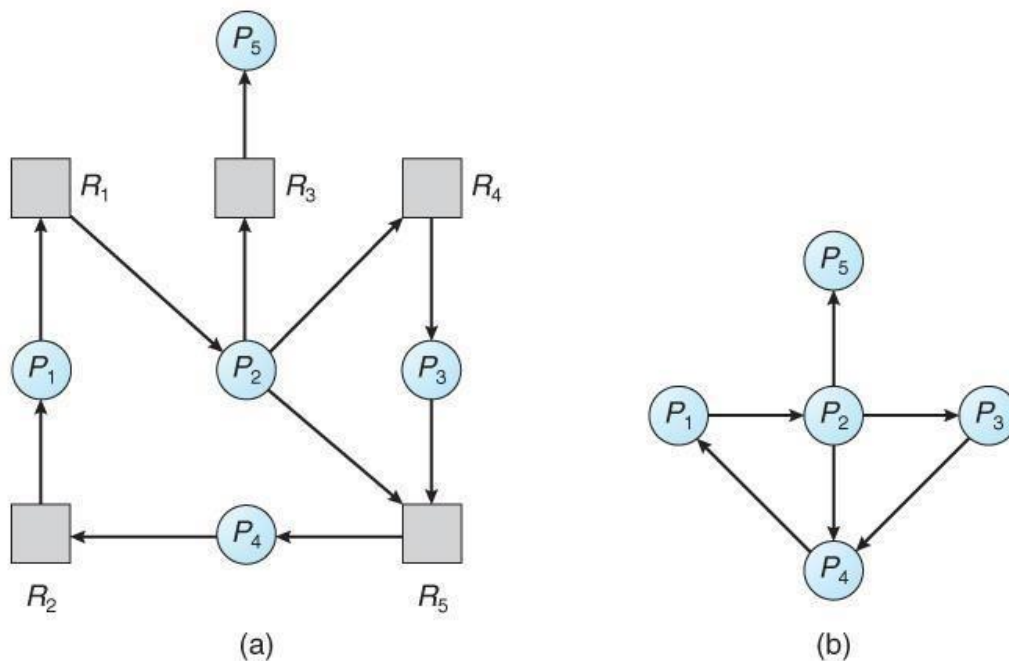
The safe sequence is P₁, P₃, P₄, P₀, P₂

Deadlock Detection and Recovery

If deadlocks are not avoided, then another approach is to detect when they have occurred and recover somehow. Constantly check for deadlocks, a policy / algorithm must be in place for recovering from deadlocks, and there is potential for lost work when processes must be aborted or have their resources preempted.

Deadlock Detection for single instance resource

- If each resource category has a single instance, then we can use a variation of the resource-allocation graph known as a *wait-for graph*.
- A wait-for graph can be constructed from a resource-allocation graph by eliminating the resources and collapsing the associated edges, as shown in the figure below.
- An arc from P_i to P_j in a wait-for graph indicates that process P_i is waiting for a resource that process P_j is currently holding.



- As before, cycles in the wait-for graph indicate deadlocks.
- This algorithm must maintain the wait-for graph, and periodically search it for cycles.

Deadlock detection for Several Instance Resources

- The detection algorithm outlined here is essentially the same as the Banker's algorithm,

with two subtle differences:

- In step 1, the Banker's Algorithm sets $\text{Finish}[i]$ to false for all i . The algorithm presented here sets $\text{Finish}[i]$ to false only if $\text{Allocation}[i]$ is not zero. If the currently allocated resources for this process are zero, the algorithm sets $\text{Finish}[i]$ to true. This is essentially assuming that IF all of the other processes can finish, then this process can finish also. Furthermore, this algorithm is specifically looking for which processes are involved in a deadlock situation, and a process that does not have any resources allocated cannot be involved in a deadlock, and so can be removed from any further consideration.
- Steps 2 and 3 are unchanged
- In step 4, the basic Banker's Algorithm says that if $\text{Finish}[i] == \text{true}$ for all i , that there is no deadlock. This algorithm is more specific, by stating that if $\text{Finish}[i] == \text{false}$ for any process P_i , then that process is specifically involved in the deadlock which has been detected.

Consider, for example, the following state, and determine if it is currently deadlock:

Process	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	0	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Now suppose that process P2 makes a request for an additional instance of type C, yielding the state shown below. Is the system now deadlocked?

Process	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C

P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	0	0	0	1			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

There occurs a deadlock because the resource B is hold by process P1 does not allow P3 to complete and hence the resources required for the other processes is not release and hence P4, P2 also cannot complete, so there occurs a deadlock.

Deadlock Recovery

After detecting the deadlock, the system can use anyone of the following procedure to recover from the deadlock.

There are two basic approaches to recovery from deadlock:

- 1) Terminate one or more processes involved in the deadlock
- 2) Preempt resources

Process Termination

Two basic approaches, both of which recover resources allocated to terminated processes:

- Terminate all processes involved in the deadlock. This definitely solves the deadlock, but at the expense of terminating more processes than would be absolutely necessary.
- Terminate processes one by one until the deadlock is broken. This is more conservative, but requires doing deadlock detection after each step.

In the latter case there are many factors that can go into deciding which processes to terminate next:

1. Process priorities.
2. How long the process has been running, and how close it is to finishing.

3. How many and what type of resources is the process holding.
4. How many more resources does the process need to complete?
5. How many processes will need to be terminated?
6. Whether the process is interactive or batch.

Resource Preemption

When preempting resources to relieve deadlock, there are three important issues to be addressed:

- **Selecting a victim** - Deciding which resources to preempt from which processes involves many of the same decision criteria outlined above.
- **Rollback** - Ideally one would like to roll back a preempted process to a safe state prior to the point at which that resource was originally allocated to the process. Unfortunately, it can be difficult or impossible to determine what such a safe state is, and so the only safe rollback is to roll back all the way back to the beginning.
- **Starvation** - How do you guarantee that a process won't starve because its resources are constantly being preempted? One option would be to use a priority system, and increase the priority of a process every time its resources get preempted. Eventually it should get a high enough priority that it won't get preempted any more.